

Task Block 2 - Multi-Agent System Report

Kopilkin Multi-Agent Financial Assistant

Mubarakov Islam

1. Project overview

Kopilkin is a mobile-first personal finance web application. Users can register, log in, add income and expense transactions, create savings goals, and ask an AI assistant questions about their spending. For Task Block 2, the project was extended with a multi-agent AI system that helps users understand where their money goes and gives practical saving advice based on real transaction data.

2. Agent specification

The assistant is an integrated multi-agent system, not a set of disconnected bots. It uses routing and chaining between specialized agents.

Agent	Responsibility
Orchestrator Agent	Coordinates memory search, routing, agent execution, response creation, and memory saving.
Router Agent	Chooses analyst or advisor path.
Analyst Agent	Retrieves real transaction summary and explains spending patterns using actual numbers.
Advisor Agent	Uses analyst output and gives practical data-grounded saving advice.

3. LLM engine and model comparison

Ollama was selected as the local LLM engine because it is simple to install, supports local open-source models, and provides an HTTP API for FastAPI integration. The instructor did not require a specific model; the requirement was to compare lightweight local models.

Criteria	llama3.2:3b	qwen2.5:3b	gemma3:4b
Context understanding	General topic understood	Wrong: assumed user owns a cafe	Correct: personal expenses
Russian quality	Mixed languages, unstable phrasing	Good Russian, wrong context	Good Russian
Usefulness	Low	Low	High
Hallucinations	Present	Present	Minimal in this test
Final decision	Not selected	Not selected	Selected

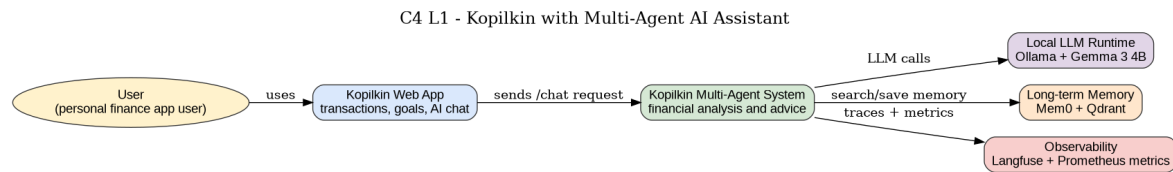
Final choice: gemma3:4b. It gave the best balance between context understanding, Russian language quality, useful financial advice, and low hallucination tendency.

Сначала по сравнению моделей — вот итог:			
	llama3.2:3b	qwen2.5:3b	gemma3:4b
Понял контекст	✓	✗ (решил что ты владелец кафе)	✓
Русский язык	✗ (мешанина)	✓	✓
Полезность	слабо	слабо	хорошо
Галлюцинации	есть	есть	минимум
Победитель — gemma3:4b. Это и напишем в отчёте с реальными примерами.			

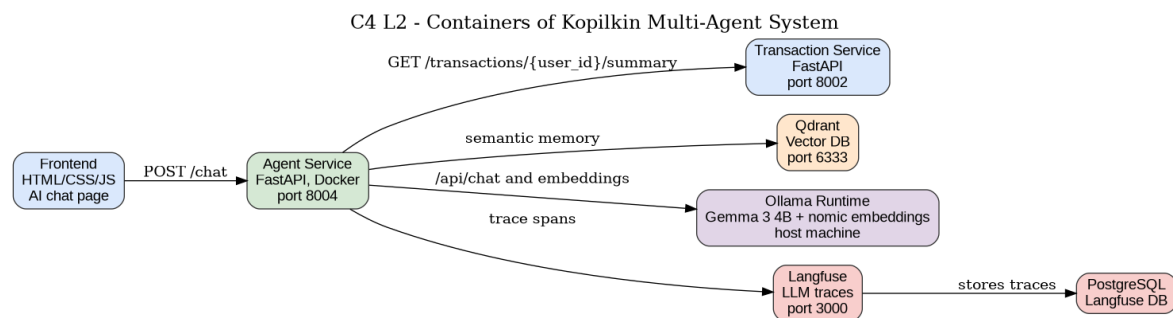
4. Multi-agent architecture and diagrams

Main flow: User -> Frontend AI Chat -> Agent Service /chat -> Orchestrator -> Memory Search -> Router Agent -> Analyst Agent -> Advisor Agent if advice is needed -> Memory Save -> Response.

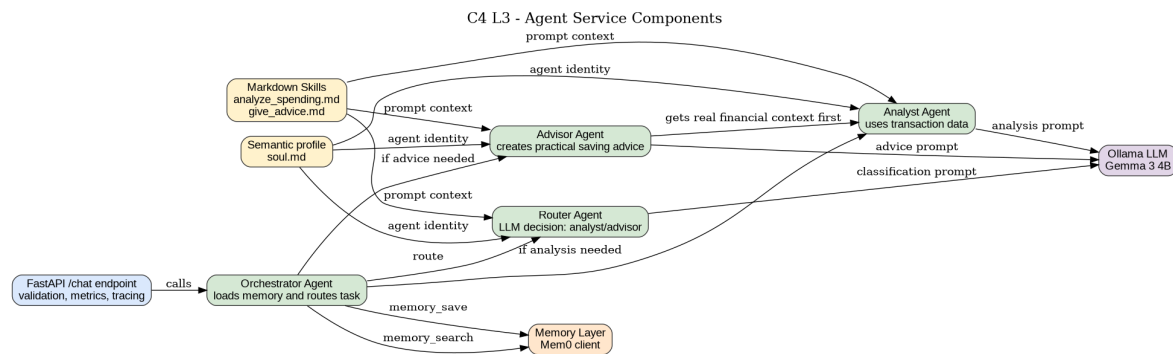
C4 L1 - Kopilkin with Multi-Agent AI Assistant



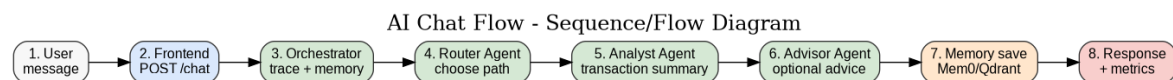
C4 L2 - Multi-Agent System Containers



C4 L3 - Agent Service Components



AI Chat Flow / Sequence Diagram



5. Isolation strategy

The agent-service, Qdrant, Langfuse, and Langfuse PostgreSQL database are containerized with Docker Compose. Ollama remains on the host machine and is accessed through host.docker.internal:11434. This hybrid setup is practical for student hardware because model execution depends on local model files and host resources.

Option	Decision
No isolation	Rejected: not reproducible and unsafe for dependencies.
Python venv	Not enough: does not isolate databases/services.

Docker containers	Selected: lightweight and reproducible.
Full VM	Too heavy for local student hardware.
Kubernetes	Future option, not necessary for local MVP.

6. Prompts, skills, and semantic Markdown

System prompts are implemented in agent-service/app/prompts/system_prompts.py. Skills are defined in agent-service/app/skills/analyze_spending.md and give_advice.md. The semantic identity file soul.md defines the role, character, limitations, and language behavior. A skill_loader.py file was added so these Markdown files can be loaded into prompt context.

7. Memory management

The system uses Mem0 + Qdrant + Ollama embeddings. Mem0 manages memory search/save operations, Qdrant stores vectors in the kopilkin_memory collection, and Ollama nomic-embed-text is used for embeddings. Memory is filtered by user_id.

Option	Advantages	Drawbacks	Decision
Classic RAG	Simple document retrieval	Not ideal for evolving personal memory	Not selected
Mem0	Easy user/session memory	Less explainable than graph memory	Selected
Zep	Good conversation memory	More backend setup	Considered
Letta / MemGPT	Advanced stateful agents	Too complex for MVP	Considered
Graphiti	Temporal knowledge graph	Heavier implementation	Future improvement
LlamaIndex	Strong retrieval framework	More document-oriented	Considered

Classic RAG is not optimal here because Kopilkin needs evolving user-specific memory across conversations, not only retrieval from static documents.

8. Evaluations

The project includes evals/test_cases.json, evals/run_evals.py, and evals/README.md. The runner sends predefined prompts to /chat and saves a JSON report.

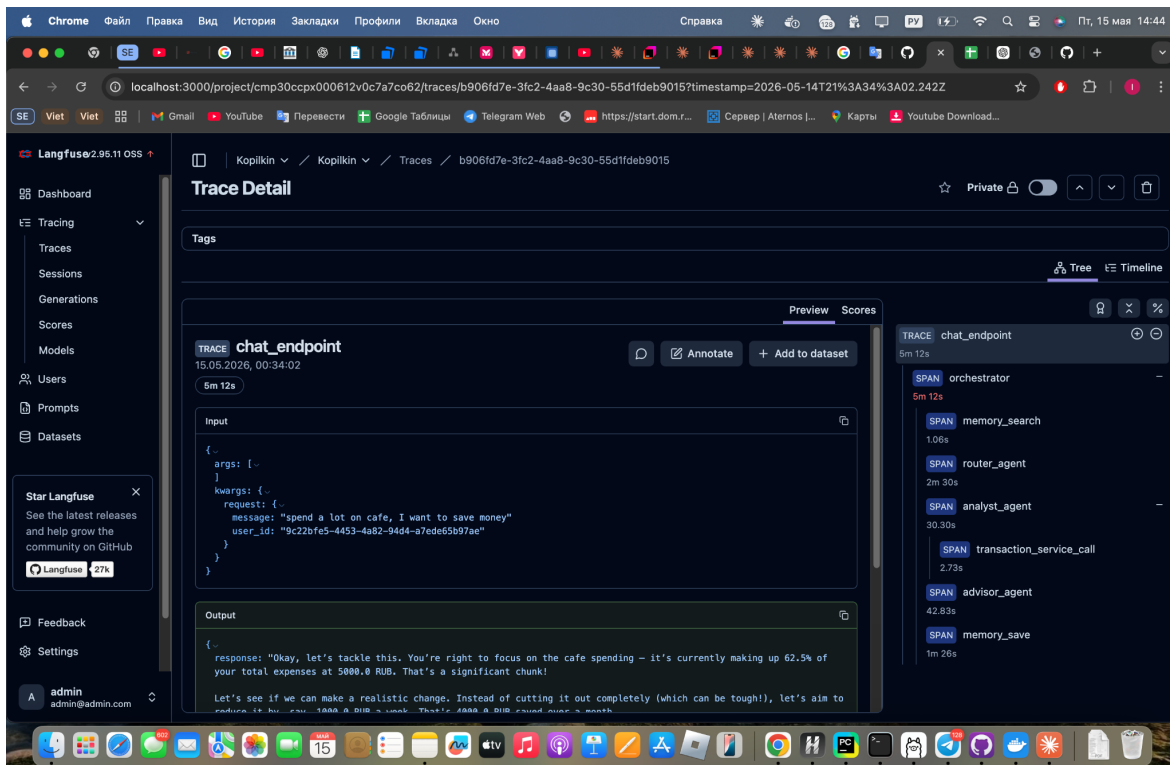
Metric	Meaning
Routing accuracy	Whether the orchestrator selects Analyst/Advisor correctly.
Groundedness	Whether the answer references real transaction data.
Hallucination rate	Whether the model invents non-existing facts or numbers.
Language consistency	Whether response language follows user language.
Tool success rate	Whether Transaction Service calls succeed.
Memory relevance	Whether retrieved memories are useful.
Latency	How long /chat takes.
Error rate	Percentage of failed chat requests.

9. Observability

Langfuse traces the chat endpoint, orchestrator, memory search, router agent, analyst agent, transaction service call, advisor agent, and memory save. The Agent Service also exposes a Prometheus-compatible /metrics endpoint and structured logs.

Implemented item	Status
Langfuse traces	Implemented and working.
/metrics endpoint	Implemented in agent-service.
Structured logs	Implemented in agent-service.
Full alerting	Planned improvement.

Evidence screenshot: Langfuse trace detail.



11. Conclusion

The Kopilkin project now includes a working multi-agent AI system with local LLM execution, Docker-based isolation, long-term memory, semantic skills, system prompts, LLM comparison, evaluation files, and observability through Langfuse and metrics. The selected model is gemma3:4b because it performed best in local tests. The current implementation is suitable for MVP demonstration and oral defense, while production improvements would include stronger alerting, persistent databases, stronger secret management, and more advanced memory evaluation.